

Relatório PBL - Jogo Tetris

Gabriel Oliveira de Freitas

Departamento de Exatas (DEXA) – Universidade Estadual de Feira de Santana (UEFS)
Av. Transnordestina, s/n - Novo Horizonte - CEP 44036-900
Feira de Santana - Bahia - Brasil

goliveira0403@gmail.com

Abstract. *This report describes the development of a version of the classic game Tetris in Python, meeting the specifications of a Blizzard Entertainment selection process. In the game, the player controls the falling of pieces, arranging them on the board to complete horizontal lines and earn points. The development improved a Problem-Based Learning methodology, and the result was a functional system that manages the delivery of pieces, scoring and game ending conditions.*

Resumo. *Este relatório descreve o desenvolvimento de uma versão do clássico jogo Tetris em Python, atendendo às especificações de um processo seletivo da Blizzard Entertainment. No jogo, o jogador controla a queda de peças, organizando-as no tabuleiro para completar linhas horizontais e ganhar pontos. O desenvolvimento seguiu uma metodologia de Aprendizagem Baseada em Problemas, e o resultado foi um sistema funcional que gerencia a movimentação das peças, pontuação e as condições de término do jogo.*

1. Introdução

A Blizzard Entertainment, uma das maiores empresas de desenvolvimento de jogos do mundo, frequentemente busca jovens talentos para compor seu time criativo e inovador. Neste ano, a empresa abriu uma oportunidade de estágio que é uma boa oportunidade para programadores iniciantes demonstrarem suas habilidades. Como parte do processo seletivo, os candidatos deveriam desenvolver uma versão do Tetris, utilizando a linguagem Python, sendo proibido a utilização de qualquer biblioteca, ferramenta, framework ou game engine python relacionado ou direcionado a produção de jogos, como o pygame.

O Tetris, criado em 1984 pelo engenheiro soviético Alexey Pajitnov, é um dos jogos mais icônicos e duradouros da história dos videogames. Nele, o jogador manipula peças de diferentes formatos, conhecidas como "Tetriminos", que caem continuamente do topo do tabuleiro. O objetivo é organizar essas peças para completar linhas horizontais, que são removidas, liberando espaço no tabuleiro e acumulando pontos para o jogador. O projeto exigiu a implementação de mecânicas fundamentais, como a movimentação das peças para os lados, rotação, além de uma peça representada por uma bomba, capaz de limpar uma área do tabuleiro.

Esse relatório está dividido em 4 tópicos: 1) Introdução: Descrição e explicação do problema e do sistema solicitado para sua resolução. 2) Metodologia: Descrição das sessões, requisitos e descrição de alto nível, ordem de codificação e ficha técnica. 3) Resultados e discussões: Manual de uso, dados de entrada e dados de saída, testes e erros. 4) Conclusão: Descrição das conclusões do que foi feito.

2. Metodologia

Durante as semanas de desenvolvimento do projeto, foi aplicada a metodologia Problem-Based Learning (PBL), onde foram discutidas e implementadas diferentes abordagens para a solução do problema. Esse processo foi dividido em várias etapas para estruturar o desenvolvimento do jogo Tetris em Python, com o objetivo de implementar as principais funcionalidades do jogo de forma eficiente.

2.1. Sessões Tutoriais

O desenvolvimento seguiu cinco sessões tutoriais, nas quais o grupo debateu ideias e analisou requisitos. Cada sessão contou com as funções: coordenador, que liderou a discussão, o secretário de quadro registrou as ideias e metas discutidas, e o secretário de mesa ficou responsável pelo relatório da sessão. Essas sessões foram fundamentais para definir a estrutura do jogo e solucionar problemas, como a inserção das peças no tabuleiro, sua descida, movimentação, rotação e verificação de colisões com bordas e outras peças.

2.2. Requisitos do Sistema

Para atender às especificações do projeto e ao modelo solicitado, foram definidos os seguintes requisitos principais para o sistema:

- **Tabuleiro:** Um tabuleiro de 10 colunas por 20 linhas;
- **Peças do Jogo:** As peças devem ser nos formatos: linha, quadrado, T, Z, S, L e J, além de uma peça especial (bomba) que limpa uma área 3x3 do tabuleiro ao ser posicionada.
- **Movimentação e Rotação:** Implementação de controles para mover as peças para a esquerda, direita, para baixo, e realizar rotações.
- **Pontuação:** O sistema registra a pontuação total do jogador, que aumenta conforme linhas são completadas e removidas. Linhas simultâneas removidas acumulam mais pontos.
- **Condições de Fim de Jogo:** O jogo termina se uma nova peça não puder ser inserida no topo do tabuleiro.

2.3. Descrição do Algoritmo

O algoritmo foi desenvolvido com foco na modularidade, utilizando funções (def), o que permitiu organizar o código em blocos independentes e reutilizáveis.

O código utiliza as seguintes bibliotecas:

- **random:** Usada para selecionar peças aleatoriamente durante o jogo, garantindo variação na sequência de peças que caem no tabuleiro.
- **os:** Destinada a limpar a tela, a fim de atualizar o tabuleiro em tempo real.
- **time:** Utilizada para controlar o tempo de atualização do jogo, determinando a velocidade de queda das peças.

- **keyboard:** Biblioteca de terceiros que captura os comandos do teclado, permitindo ao usuário mover e rotacionar as peças.

Para a representação do tabuleiro do Tetris, foram utilizadas listas de listas (ou seja, uma matriz 2D), onde cada sublista representa uma linha do tabuleiro, e cada elemento dentro da sublista representa um elemento da matriz.

As peças do jogo também foram representadas como listas de listas, em que cada peça é uma matriz contendo o formato e as cores específicas da peça, como é possível ver a seguir:

```
#Lista de peças do jogo com a bomba.
peças = [
    [[' ', ' ', ' ', ' ', ' '], #Linha (azul).
    [[' ', ' ', ' ', ' ', ' '], #Quadrado (amarelo).
    [[' ', ' ', ' ', ' ', ' '], #T (roxo).
    [[' ', ' ', ' ', ' ', ' '], #Z (vermelho).
    [[' ', ' ', ' ', ' ', ' '], #S (verde).
    [[' ', ' ', ' ', ' ', ' '], #L (laranja).
    [[' ', ' ', ' ', ' ', ' '], #J (marrom).
    [[' ', ' ', ' ', ' ', ' '], #Bomba.
]
```

Figura 1. Peças coloridas

Abaixo estão as principais funções implementadas no código:

- **mostrar_tabuleiro:** Função que itera sobre a matriz do tabuleiro, exibindo cada elemento da matriz e atualizando a pontuação total na tela. Essa função é chamada repetidamente para mostrar o estado atual do jogo.
- **inserir_peça e remover_peça:** Funções responsáveis por inserir ou remover uma peça no tabuleiro, acessando diretamente a matriz para posicionar ou limpar os elementos da matriz ocupados pela peça. Essa abordagem permite manipular a posição das peças no tabuleiro de forma precisa e eficiente.
- **verificar_colisao:** Função que verifica se a peça em movimento está colidindo com outra peça ou com as bordas do tabuleiro. Ela percorre a matriz do tabuleiro e a matriz da peça, retornando “True” se encontrar colisão, ou “False” caso contrário.
- **girar_peça:** Rotaciona uma peça em 90 graus no sentido horário, criando uma nova matriz com a largura e altura invertidas. A função analisa os elementos da peça original para a matriz rotacionada, retornando a peça rotacionada para uso no tabuleiro.
- **explodir_bomba:** Função que simula uma explosão de uma área 3x3 ao redor da peça "bomba". A função percorre uma pequena matriz ao redor da posição da bomba e redefine os valores desses elementos da matriz como vazios.
- **quebrar_linhas:** Função que identifica e remove linhas completas, percorrendo a matriz do tabuleiro. Linhas sem elementos vazios são deletadas, novas linhas

vazias são inseridas no topo do tabuleiro e tudo que estava acima da linha quebrada desce. A função também ajusta a pontuação com base no número de linhas removidas simultaneamente.

- **descer_peça:** Função principal que controla a movimentação e a descida das peças. Esta função utiliza loops e condicionais para mover as peças pela matriz do tabuleiro, permitindo deslocamento para os lados, rotação e aumento da velocidade de queda. A velocidade é ajustada conforme a dificuldade escolhida pelo usuário.
- **reiniciar_jogo:** Esta função reinicia o tabuleiro e a pontuação total do jogo, limpando a matriz do tabuleiro ao substituir cada elemento da matriz por um elemento vazio e redefinindo a pontuação para zero. É importante para permitir que o jogador comece uma nova partida após o fim do jogo.
- **tetris:** Função que serve como menu principal do jogo, onde o jogador escolhe entre iniciar uma nova partida e selecionar a dificuldade, consultar o manual do jogo, ou encerrar o programa. Esta função controla o loop principal, chamando “descer_peça” para iniciar o jogo e reiniciando o tabuleiro após o término de cada partida.

O uso de listas, matrizes e funções foi fundamental para estruturar o jogo, garantindo o controle do tabuleiro e das peças, além de facilitar operações como colisão e rotação.

2.4. Ordem de Codificação

O desenvolvimento do código foi estruturado em seis semanas, com metas definidas a cada etapa para organizar a implementação das funcionalidades essenciais do jogo Tetris.

- **Primeira Semana:** Foi realizada apenas a criação do tabuleiro.
- **Segunda Semana:** Foi feita a implementação do movimento das peças, sua rotação, e o uso do módulo random para selecionar as peças aleatoriamente. Nessa fase, também foi iniciada a inserção das peças no tabuleiro.
- **Terceira Semana:** Foi adicionada a funcionalidade de queda automática das peças e o controle do seu movimento lateral. Foram introduzidos métodos para verificação de colisões, permitindo que as peças fossem limitadas pelas bordas do tabuleiro e por outras peças.
- **Quarta Semana:** Nessa fase, foram implementadas as funções de rotação das peças e de limpeza de linhas completas e a descida do que estava em cima da linha que foi quebrada.
- **Quinta Semana:** Semana dedicada à implementação das funcionalidades de pontuação e da bomba. A função de pontuação foi configurada para ajustar a quantidade de pontos com base no número de linhas removidas simultaneamente. Já a bomba foi programada para limpar uma área de 3x3 ao redor de onde cair.
- **Sexta Semana:** A última semana foi destinada a ajustes finais, testes de integração e correção de erros. Todas as funcionalidades foram revisadas para

garantir estabilidade e eliminar possíveis falhas no código. Esta semana serviu para refinar o código e assegurar que o jogo estivesse pronto para uso.

2.5. Ficha Técnica

O sistema operacional utilizado para o desenvolvimento do programa foi o Windows 11 Home Single Language. A linguagem de programação escolhida foi Python, versão 3.12.4. A IDE (Ambiente de Desenvolvimento Integrado) escolhida foi o Visual Studio Code (VSCode), versão 1.93.0. Além disso, foi necessário baixar a biblioteca “keyboard” pelo comando “pip install keyboard” no terminal de comando.

3. Resultados e Discussões

O desenvolvimento do jogo Tetris em Python resultou em um sistema funcional que atende aos requisitos propostos, permitindo a manipulação e queda contínua de peças, além de mecanismos de pontuação e um efeito especial para a bomba. A seguir, são apresentados os detalhes de uso, testes realizados e desafios superados durante a implementação.

3.1. "Manual" de Uso

Antes de executar o código é necessário baixar a biblioteca “keyboard” pelo comando “pip install keyboard” no terminal de comando. Ao executar o código, será apresentado ao usuário três opções no menu:

- ENTER - INICIAR JOGO
- Digite “2” para ver o manual do jogo
- Digite “3” para fechar o jogo

Pressionando a tecla “enter”, é solicitado que o usuário escolha o nível de dificuldade (1 para fácil, 2 para médio e 3 para difícil), o que ajusta a velocidade de queda das peças. Durante o jogo, as teclas permitem:

- **W** ou ↑: Rotacionar a peça
- **A** ou ←: Mover a peça para a esquerda
- **D** ou →: Mover a peça para a direita
- **S** ou ↓: Acelerar a queda da peça

Quando a bomba cai no tabuleiro, ela elimina todos os elementos em uma área de 3x3 ao seu redor, limpando qualquer peça próxima dentro desse alcance.

3.2. Dados de Entrada e Dados de Saída

Dados de Entrada: Escolha da dificuldade, comandos de movimentação, rotação e aceleração.

Dados de Saída: Atualização visual do tabuleiro, pontuação acumulada e mensagem de "Game Over".

3.3. Testes

Vários testes foram realizados para assegurar o funcionamento correto das funcionalidades do jogo:

- **Movimentação e Colisão das Peças:** Foi verificado que as peças respeitam as bordas do tabuleiro e não se sobrepõem.
- **Rotação das Peças:** Testou-se a rotação em diferentes posições para garantir que as peças giram corretamente sem ultrapassar os limites do tabuleiro.
- **Quebra de Linhas e Pontuação:** Foi testado se as linhas quebravam simultaneamente ao serem completadas e se a pontuação estava sendo contabilizada corretamente.
- **Ação da Bomba:** A bomba foi testada para garantir que elimina elementos em uma área de 3x3 ao seu redor.
- **Condições de Fim de Jogo:** Testou-se a condição de término quando não há espaço para uma nova peça, exibindo "Game Over".

3.4. Erros

O código apresenta poucos erros, há um leve atraso ao mover as peças nas dificuldades média e fácil, que faz com que o usuário tenha que manter pressionada a tecla para movimentar a peça e evitar esse atraso. Além disso, ao utilizar as teclas “W”, “A”, “S” e “D” durante o jogo, essas letras ficam registradas no terminal ao final da partida, exigindo que o usuário apague esses caracteres antes de inserir novos comandos.

4. Conclusão

O projeto atendeu aos requisitos propostos, implementando as funcionalidades básicas do Tetris e adicionando a bomba para trazer mais recursos para o jogo. A organização do código em módulos permitiu clareza e eficiência na implementação, favorecendo a manutenção e possíveis expansões futuras.

Apesar do sucesso na implementação das funcionalidades principais, algumas limitações foram identificadas, como o atraso na resposta ao movimento das peças em dificuldades mais baixas e a permanência de teclas pressionadas no terminal após o término do jogo. Essas questões, embora não comprometam o funcionamento geral, representam oportunidades para aprimoramento.

A metodologia Problem-Based Learning (PBL) desempenhou um papel crucial no aprendizado de programação, proporcionando uma situação prática que permitiu ao desenvolvedor do jogo ganhar conhecimento e desenvolver habilidades essenciais para um programador. O desenvolvimento do código foi uma oportunidade significativa para a prática de conceitos fundamentais, como criação e uso de funções, manipulação de listas e implementação de matrizes para organizar o tabuleiro e as peças.

5. Bibliografia

SEVERANCE, Charles. "Aprenda Computação com Python." Editora Pearson, 2016.

MATTHES, Eric. "Python Crash Course." No Starch Press, 2019.

Funções em Python: Como usar a função def. Disponível em:
<https://www.devmedia.com.br/funcoes-em-python/37340>. Acesso em: 22/09 set. 2024